OBSERVABILITY METRICS

Observability metrics are measurements that help you understand what is happening inside your system (application, server, Kubernetes cluster, etc.) without directly looking inside the code.

It helps teams:

- Detect failures quickly
- Troubleshoot production issues
- Monitor performance
- Improve reliability

Observability is built on three pillars:

1. Metrics
2. Logs
3. Traces

1 Metrics

Metrics are **numerical values measured over time**.

They help answer:

- Is the system healthy?
- Is CPU high?
- Is memory increasing?
- How many requests are failing?

1) Application Metrics

Application metrics measure the performance and behavior of the application itself.

2) System Metrics

System metrics measure infrastructure and resource utilization.

3) Business Metrics

Business metrics measure how the system impacts business outcomes.

For instance in kubernetes,

Node-Level Metrics

Metric	Meaning
`node_cpu_seconds_total`	CPU usage
`node_memory_MemAvailable_bytes`	Available memory

Pod-Level Metrics

Metric	Meaning
`kube_pod_status_phase`	Running / Pending / Failed

2 Logs

Logs are timestamped entries of specific events generated by systems, applications, networks, and infrastructure.

They help answer:

- Why did this error happen?
- What exactly failed?
- What input caused the issue?

1)System logs: These include events like connection attempts, errors, and configuration changes.

2)Application logs: They record software changes, CRUD operations, application authentication, and other events to help diagnose issues.

3)Network logs: Network logs record data from events that take place on a network or device, including network traffic, security events, and user activity.

3 Traces

Traces are telemetry signals that let engineers see applications and services from a user-session perspective. It track a request across multiple services **a** in a distributed system.

They help answer:

- Where is the request slow?
- Which microservice is causing delay?
- What path did the request follow?
- Which dependency is failing?

Each trace consists of:

- Trace ID (unique request identifier)
- Multiple spans (individual operations)
- Parent-child relationships between spans

4 Profiles

Profiling analyzes how an application uses:

- CPU
- Memory
- Threads
- Garbage collection
- Function execution time

It helps identify performance bottlenecks inside the code.

The 4 golden signals for SRE teams:

The Four Golden Principles (also known as the Four Golden Signals) are key metrics used to monitor and ensure system reliability and performance.

1️⃣ Latency

Latency measures the time taken to process a request. High latency indicates slow system performance and can negatively impact user experience.

2️⃣ Traffic

Traffic represents the demand on the system, such as the number of requests per second. Monitoring traffic helps in capacity planning and scaling decisions.

3️⃣ Errors

Errors track the rate of failed requests (e.g., HTTP 4xx and 5xx responses). A high error rate indicates application or dependency failures.

4️⃣ Saturation

Saturation measures how "full" the system is, typically through CPU, memory, disk, or network utilization. High saturation indicates the system is nearing its limits.